

Vitamin Helix

Infrastructure & Deployment Reference

Architecture Overview

Vitamin Helix runs on a self-hosted local network setup within the Vitamin Dubai office. The system is composed of three physical nodes: a development laptop, a Mini-PC server, and a Synology NAS — all on the same LAN.

Node	Role	OS / Software
Windows Laptop	Development & Testing	Windows / VS Code / Python / Flask
Mini-PC (Server Room)	Production App Server	Linux / Flask / PostgreSQL 18
Synology NAS	File Storage & DB Backups	Synology DSM / SMB / VPN

Network Flow

[Windows Laptop] → git push → [GitHub]
↓ git pull (SSH)
[Mini-PC / Linux – Server Room]
Helix Flask App | PostgreSQL
↓ pg_dump (cron, daily)
[Synology NAS – Admin/Database/]
VPN tunnel for remote file access
Synology DSM for NAS management

Remote Access

Mini-PC	AnyDesk installed — use for remote desktop access if maintenance needed outside office
Mini-PC	SSH available on LAN for fast terminal access and deployments
Synology NAS	Managed via Synology DSM (browser-based). No AnyDesk needed — DSM handles remote admin
NAS Files	VPN tunnel for secure remote access to files from outside the network

Deployment Workflow

Every update follows this flow from dev laptop to production Mini-PC:

1. Develop & test

Write code on the Windows laptop. Test locally with Flask dev server.

2. Commit & push

```
git commit -m "description"
git push
```

3. SSH into Mini-PC

```
ssh ezekiel@
```

4. Pull & restart

```
cd ~/project-tracker
git pull
sudo systemctl restart helix
```

5. Schema changes

If any migration scripts are needed, run them manually after pull:

```
python add_example_column.py
```

Tip: One-liner deploy from your laptop: ssh ezekiel@ "cd ~/project-tracker && git pull && sudo systemctl restart helix"

Versioning & Changelog

During the build phase, all changes are logged in CHANGELOG.txt at the project root. Each session's changes are appended as a new dated entry.

Build phase	Append to CHANGELOG.txt in the project root after each session
Go-live (v1.0)	Tag the release in git and create a GitHub Release with the changelog as release notes
Post go-live	Each meaningful update gets a version tag (v1.1, v1.2) and a GitHub Release entry
Rollback	git checkout <tag> reverts code to any prior release instantly

Tagging convention: git tag v1.0 → git push origin v1.0 — only at release points, not on every commit.

Backup Strategy

Daily snapshot	pg_dump runs via cron on Mini-PC. Output saved to NAS: Admin/Database/daily/
Weekly snapshot	Full pg_dump every Sunday saved to NAS: Admin/Database/weekly/
Retention	Keep last 30 daily dumps. Cron auto-deletes older files.
Before deploy	Manual pg_dump recommended before any major update (save to NAS)
NAS location	Synology NAS → Admin / Database /

Daily cron job (Mini-PC)

```
0 2 * * * pg_dump -U postgres -d project_tracker -F c -f
/mnt/nas/Admin/Database/daily/$(date +%Y-%m-%d).dump
0 2 * * 0 pg_dump -U postgres -d project_tracker -F c -f
/mnt/nas/Admin/Database/weekly/$(date +%Y-%m-%d).dump
```

Initial Migration to Mini-PC

One-time setup to move from dev laptop to the Mini-PC production server:

1. Install Linux on Mini-PC
2. Install PostgreSQL 18, Python 3.x, pip, git

- 3. Clone repo: `git clone`
- 4. Install dependencies: `pip install -r requirements.txt`
- 5. On dev laptop – export DB: `pg_dump -U postgres -d project_tracker -F c -f helix_launch.dump`
- 6. Copy `helix_launch.dump` to Mini-PC (USB or via NAS share)
- 7. On Mini-PC – create DB: `createdb -U postgres project_tracker`
- 8. Restore: `pg_restore -U postgres -d project_tracker -F c helix_launch.dump`
- 9. Set up `.env` / config with correct DB credentials
- 10. Run app and verify everything works
- 11. Set up `systemd` service so app starts on boot
- 12. Install AnyDesk
- 13. Mount NAS share and configure backup cron jobs
- 14. Tag as `v1.0` and create GitHub Release

Health & Monitoring

Health endpoint	Add /ping route to Helix returning HTTP 200. Used for uptime checks.
Uptime check	Cron on Mini-PC checks /ping every 5 mins. Sends email/notification if down.
Disk alert	Monitor Mini-PC disk usage — alert if above 80% capacity.
DB size	Periodic check on project_tracker DB size to anticipate storage needs.

Security Notes

Network exposure	Helix bound to LAN only — not exposed to the public internet
Remote access	VPN required to access Helix or NAS from outside the office network
PostgreSQL	Postgres listens on localhost only on Mini-PC. Strong password set.
SSH	SSH key-based auth recommended. Disable password login on Mini-PC.
Git	Repo on GitHub (private). No secrets committed — use .env for credentials.

Rollback Procedure

If a deployment breaks something:

```
# Code rollback – revert to previous version tag
git checkout v1.x
sudo systemctl restart helix

# Database rollback – restore from last NAS snapshot
pg_restore -U postgres -d project_tracker -F c
/mnt/nas/Admin/Database/daily/YYYY-MM-DD.dump
```